

MartPoint Retail v4.0.8 — Technical New Installation Guide

1. Overview

This guide walks a technical installer through a clean MartPoint Retail v4.0.8 installation on Linux, macOS or Windows. It assumes command-line/SSH and MySQL admin access.

2. Server Requirements

2.1 Minimum stack

- PHP 7.4+ (8.0/8.1 supported)
- MySQL 5.7+ or MariaDB 10.2+
- Apache with mod_rewrite or Nginx with rewrite rules
- SSL certificate for production

2.2 Required PHP extensions

- mysqli, pdo_mysql, mbstring, intl, openssl, curl, gd, zip, fileinfo, json, session

2.3 Recommended PHP settings

```
memory_limit = 256M
upload_max_filesize = 20M
post_max_size = 20M
max_execution_time = 300
max_input_vars = 3000
```

3. Pre-Installation Checklist

1. Register the client domain in your licensing/partner portal and obtain a license key.
2. Obtain the v4.0.8 source archive or clone the private repo.
3. Create the client database and database user.
4. Point the client domain to the web root.
5. Install an SSL certificate and ensure HTTPS is reachable.

4. Database Setup

4.1 Create the database

```
CREATE DATABASE martpoint CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'martpoint'@'localhost' IDENTIFIED BY 'StrongPasswordHere';
GRANT ALL PRIVILEGES ON martpoint.* TO 'martpoint'@'localhost';
FLUSH PRIVILEGES;
```

4.2 Verify connection

```
mysql -u martpoint -p martpoint -e "SHOW TABLES;"
```

Security: Never use the demo password. Keep the password only in `application/config/database.php`.

5. Source Deployment

1. Upload or clone the source into the web root, e.g. `/var/www/martpoint/`.
2. Make these directories writable by the web server:
 - `uploads/`
 - `application/cache/`
 - `application/logs/`
3. On Linux:

```
chown -R www-data:www-data /var/www/martpoint && chmod -R 755 uploads/ application/cache/ application/logs/
```

6. Configuration

6.1 `application/config/database.php`

```
$db['default'] = array(
    'hostname' => 'localhost',
    'username' => 'martpoint',
    'password' => 'StrongPasswordHere',
    'database' => 'martpoint',
    'dbdriver' => 'mysqli',
    'char_set' => 'utf8mb4',
    'dbcollat' => 'utf8mb4_unicode_ci',
    ...
);
```

6.2 `application/config/config.php`

- Set `base_url` to the client HTTPS domain.
- Generate a 32-byte encryption key and place it in `$config['encryption_key']`.
- Enable hooks: `$config['enable_hooks'] = TRUE;`
- Set the cron key: `$config['cron_secret_key'] = 'a-long-random-string';`
- Configure session driver, cookie and CSRF settings for production.

6.3 Web server rewrite

Ensure `index.php` is the default and URLs are rewritten. The repo includes an `.htaccess` for Apache. For Nginx, add the standard CodeIgniter rewrite block.

7. License Activation

1. Obtain the license key from the Avario / MartPoint partner portal or Stripe subscription dashboard.
2. During the web installer, enter the license key.

3. The installer validates the key against the licensing server. Ensure the server has internet during install (or use an offline license for air-gapped installs).
4. Ensure the license domain matches the configured `base_url`.

Note: Each production domain needs its own license. Sub-domains and `.test` domains may use development licenses depending on the partner agreement.

8. Web Installer

1. Browse to `https://client-domain.test/setup/install/index.php`.
2. Accept the license agreement and enter the license key.
3. Confirm the database credentials. The installer tests the connection.
4. Enter the super admin name, email and password.
5. Enter the business name, country, currency and industry type.
6. Click **Install**. The installer executes `setup/install/includes/db.txt` (base schema) and `setup/install/includes/db_install_extensions.sql` (v4.0.8 extensions).
7. When the installer reports success, click **Go to Login**.

9. Post-Installation Security

1. Delete or restrict the `setup/install/` directory.
2. Enable HTTPS-only cookies in `config.php` for production.
3. Disable `db_debug` in `database.php` for production.
4. Set `log_threshold` to `1` or `2`; avoid `4` in production.

10. First Login & Store Setup

1. Log in as the super admin created during install.
2. Go to **Store ? Store Settings** and upload the client logo.
3. Configure tax, payment modes, units of measure, warehouses and branches.
4. Create users and assign roles (Admin, Sales, Accounts, etc.).
5. Enable the modules the client needs in the business profile.

11. Email / SMTP Configuration

MartPoint sends emails for reports, debt reminders and subscription alerts. Emails require an SMTP or Resend provider to be configured.

1. Go to **Settings ? Email ? Email Settings**.
2. Choose the provider: **SMTP** or **Resend**.
3. For SMTP, enter the host, port, username, password and encryption (SSL/TLS).
4. Set the **From Email** and **From Name**.
5. Click **Test Email** and confirm the test arrives in the inbox (not spam).
6. Configure SPF, DKIM and DMARC DNS records for the sending domain to improve deliverability.

Security: Email credentials are stored in `db_email_settings` . Use a dedicated sending account and rotate the password regularly.

12. SMS / WhatsApp (Optional)

If the client wants SMS debt reminders or WhatsApp report messages, configure an SMS gateway.

1. Go to **Settings ? SMS**.
2. Enter the SMS API URL and auth credentials.
3. Use the placeholders `{{MOBILE}}` and `{{MESSAGE}}` in the API URL/body.
4. Test with one mobile number before enabling for customers.

13. Cron Jobs

Cron jobs are required for scheduled report emails, debt reminders and subscription expiry alerts. Without cron, these features will never run automatically.

13.1 Set the cron secret key

Add `$config['cron_secret_key']` to `application/config/config.php` before setting up cron. The default fallback is `martpoint_cron_2024` ; change it in production.

13.2 Crontab example (Linux)

```
# Scheduled reports (daily at 8:00 AM)
0 8 * * * curl -s "https://client-domain.test/cron/run_scheduled_reports?key=YOUR_SECRET_KEY" > /dev/null

# Debt reminders (daily at 9:00 AM)
0 9 * * * curl -s "https://client-domain.test/cron/send_debt_reminders?key=YOUR_SECRET_KEY" > /dev/null

# Subscription reminders (daily at 10:00 AM)
0 10 * * * curl -s "https://client-domain.test/subscription_license/run_reminders" > /dev/null 2>&1
```

13.3 CLI alternative

```
php index.php cron run_scheduled_reports YOUR_SECRET_KEY
php index.php cron send_debt_reminders YOUR_SECRET_KEY
php index.php subscription_license run_reminders
```

Security: The `subscription_license/run_reminders` endpoint has no built-in key. Restrict it by IP in `.htaccess` or Nginx, or run it only from the server itself.

14. Payment Gateways (Optional)

If the client will receive online payments, enable the relevant gateways.

1. Go to **Settings ? Payment Gateways**.
2. Enable Stripe, Paystack, Instamojo or other supported gateways.
3. Enter the live keys and webhooks URL shown in the settings.

4. Place a test transaction before going live.

15. Backups

1. Automate a daily `mysqldump` to an offsite location.
2. Back up the `uploads/` directory daily.
3. Keep `application/config/config.php` and `database.php` in a separate, secure backup.
4. Test a restore to a staging database at least once a month.

16. Verification & Go-Live

1. Check `db_sitesettings.version` equals **4.0.8**.
2. Confirm no red console errors on login, dashboard and POS.
3. Create a test item, add it to cart and complete a test sale.
4. Generate a receipt to confirm PDF/print rendering.
5. Send a test email from **Email Settings**.
6. Run each cron URL once manually and verify JSON success responses.

17. Troubleshooting

17.1 Blank / 500 error

- Check `application/logs/` for the latest log file.
- Confirm `database.php` credentials and that MySQL is reachable.
- Verify PHP version and required extensions.

17.2 Installer fails at database import

- Increase `max_allowed_packet` and `max_execution_time` in MySQL.
- Confirm the database was created with `utf8mb4` collation.
- Run the SQL files manually through phpMyAdmin or MySQL CLI.

17.3 License validation fails

- Confirm the license domain matches the `base_url`.
- Check internet connectivity from the server.
- Verify the license has not expired or been used on another domain.

17.4 Emails go to spam

- Configure SPF, DKIM and DMARC for the sending domain.
- Use a transactional email provider instead of a shared mailbox.

17.5 Cron returns 403

- Confirm the `key` query matches `cron_secret_key` in `config.php`.
- For CLI, pass the key as the third argument.

Final tip: Keep the client's `database.php` and `config.php` out of source control. Document the license key, admin credentials, cron key and email settings securely and separately.